www.ignousite.com
Course Code : MCS-208
Course Title : Data Structures and Algorithms
Assignment Number : PGDCA(2)/208/Assignment/2022-23
Maximum Marks : 100
Weightage : 40% www.ignousite.com
Last Date of Submission : 31th October, 2022 (for July sessi
: 15th April, 2023 (for January session)

Question 1: What are Sparse Matrices? Explain with example(s)

Ans. Sparse matrix: A sparse matrix is a two-dimensional data object made of m rows and n columns, therefore having total m x n values. If most of the elements of the matrix have 0 value, then it is called a sparse matrix.

Why to use Sparse Matrix instead of simple matrix

- **Storage**: There are lesser non-zero elements than zeros and thus lesser memory can be used to store only those elements.
- **Computing time**: Computing time can be saved by logically designing a data structure traversing only non-zero elements..

Example:

0 0 3 0 4
0 0 5 7 0
0 0 0 0 0
0 2 6 0 0

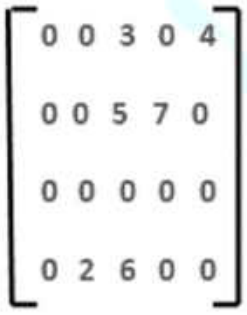
Representing a sparse matrix by a 2D array leads to wastage of lots of memory as zeroes in the matrix are of no use in most of the cases. So, instead of storing zeroes with non-zero elements, we only store non-zero elements. This means storing non-zero elements with triples- (Row, Column, value).

Sparse Matrix Representations can be done in many ways following are two common representations:

1. Array representation
2. Linked list representation

Method 1: Using Arrays: 2D array is used to represent a sparse matrix in which there are three rows named as

- **Row**: Index of row, where non-zero element is located
- **Column**: Index of column, where non-zero element is located
- **Value**: Value of the non zero element located at index – (row,column)



Row	0	0	1	1	3	3
Column	2	4	2	3	1	2
Value	3	4	5	7	2	6

Program:

```
// C++ program for Sparse Matrix Representation
// using Array
#include <iostream>
using namespace std;

int main()
{
	// Assume 4x5 sparse matrix
	int sparseMatrix[4][5] =
	{
		{0 , 0 , 3 , 0 , 4 },
		{0 , 0 , 5 , 7 , 0 },
		{0 , 0 , 0 , 0 , 0 },
		{0 , 2 , 6 , 0 , 0 }
	};

	int size = 0;
	for (int i = 0; i < 4; i++)
		for (int j = 0; j < 5; j++)
			if (sparseMatrix[i][j] != 0)
				size++;

	// number of columns in compactMatrix (size) must be
	// equal to number of non - zero elements in
	// sparseMatrix
	int compactMatrix[3][size];

	// Making of new matrix
	int k = 0;
	for (int i = 0; i < 4; i++)
		for (int j = 0; j < 5; j++)
			if (sparseMatrix[i][j] != 0)
			{
				compactMatrix[0][k] = i;
				compactMatrix[1][k] = j;
				compactMatrix[2][k] = sparseMatrix[i][j];
				k++;
			}

	for (int i=0; i<3; i++)
	{
		for (int j=0; j<size; j++)
			cout <<" "<< compactMatrix[i][j];
```

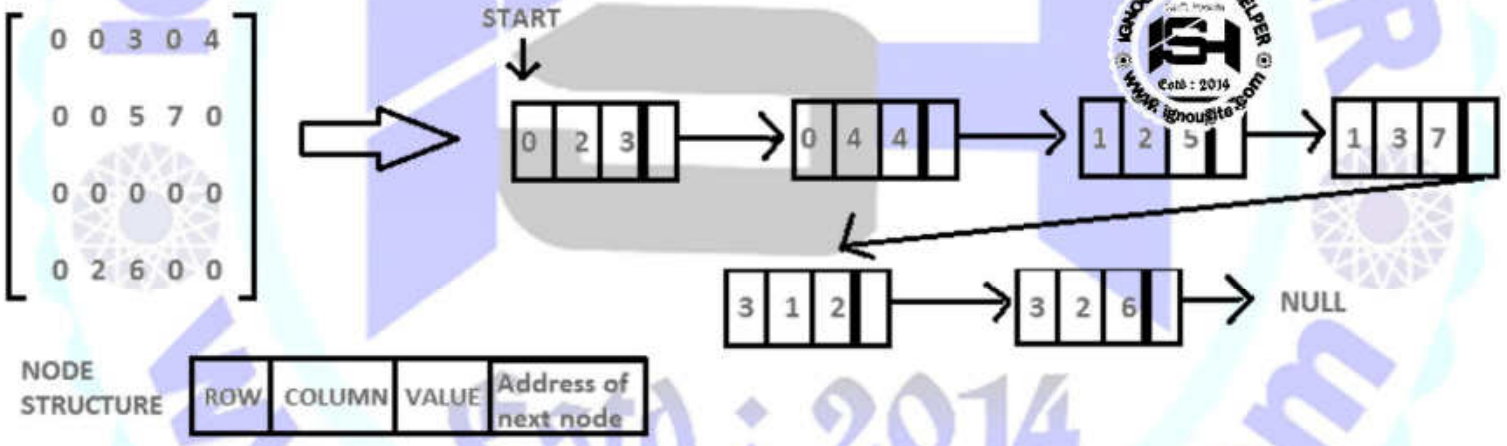
```
            cout <<"\n";
        }
        return 0;
}

// this code is contributed by ignoustudyhelper
```

Output:

```
0 0 1 1 3 3
2 4 2 3 1 2
3 4 5 7 2 6
```

Method 2: Using Linked Lists: In linked list, each node has four fields. These four fields are defined as:

- **Row**: Index of row, where non-zero element is located
- **Column**: Index of column, where non-zero element is located
- **Value**: Value of the non zero element located at index – (row,column)
- **Next node**: Address of the next node



Program:

```
// C++ program for sparse matrix representation.
// Using Link list
#include<iostream>
using namespace std;

// Node class to represent link list
class Node
{
        public:
        int row;
        int col;
```

```
        int data;
        Node *next;
};

// Function to create new node
void create_new_node(Node **p, int row_index,
                                        int col_index, int x)
{
        Node *temp = *p;
        Node *r;

        // If link list is empty then
        // create first node and assign value.
        if (temp == NULL)
        {
                temp = new Node();
                temp->row = row_index;
                temp->col = col_index;
                temp->data = x;
                temp->next = NULL;
                *p = temp;
        }

        // If link list is already created
        // then append newly created node
        else
        {
                while (temp->next != NULL)
                        temp = temp->next;

                r = new Node();
                r->row = row_index;
                r->col = col_index;
                r->data = x;
                r->next = NULL;
                temp->next = r;
        }
}

// Function prints contents of linked list
// starting from start
void printList(Node *start)
{
        Node *ptr = start;
        cout << "row_position:";
```

```
        while (ptr != NULL)
        {
                cout << ptr->row << " ";
                ptr = ptr->next;
        }
        cout << endl;
        cout << "column_position:";

        ptr = start;
        while (ptr != NULL)
        {
                cout << ptr->col << " ";
                ptr = ptr->next;
        }
        cout << endl;
        cout << "Value:";
        ptr = start;

        while (ptr != NULL)
        {
                cout << ptr->data << " ";
                ptr = ptr->next;
        }
}

// Driver Code
int main()
{

        // 4x5 sparse matrix
        int sparseMatrix[4][5] = { { 0 , 0 , 3 , 0 , 4 },
                                   { 0 , 0 , 5 , 7 , 0 },
                                   { 0 , 0 , 0 , 0 , 0 },
                                   { 0 , 2 , 6 , 0 , 0 } };

        // Creating head/first node of list as NULL
        Node *first = NULL;
        for(int i = 0; i < 4; i++)
        {
                for(int j = 0; j < 5; j++)
                {

                        // Pass only those values which
                        // are non - zero
                        if (sparseMatrix[i][j] != 0)
```

```
                    create_new_node(&first, i, j,
                                                    sparseMatrix[i][j]);
                }
        }
        printList(first);

        return 0;
}

// This code is contributed by ignoustudyhelper
```

Output:

```
row_position:0 0 1 1 3 3
column_position:2 4 2 3 1 2
Value:3 4 5 7 2 6
```

Question 2: How many different traversals of a Binary Tree are possible? Explain them with example(s).

Ans. Binary Tree Traversal in Data Structure: The tree can be defined as a non-linear data structure that stores data in the form of nodes, and nodes are connected to each other with the help of edges. Among all the nodes, there is one main node called the root node, and all other nodes are the children of these nodes.

In any data structure, traversal is an important operation. In the traversal operation, we walk through the data structure visiting each element of the data structure at least once. The traversal operation plays a very important role while doing various other operations on the data structure like some of the operations are searching, in which we need to visit each element of the data structure at least once so that we can compare each incoming element from the data structure to the key that we want to find in the data structure. So like any other data structure, the tree data also needs to be traversed to access each element, also known as a node of the tree data structure.

There are different ways of traversing a tree depending upon the order in which the tree's nodes are visited and the types of data structure used for traversing the tree. There are various data structures involved in traversing a tree, as traversing a tree involves iterating over all nodes in some manner.

As from a given node, there could be more than one way to traverse or visit the next node of the tree, so it becomes important to store one of the nodes traverses further and store the rest of the nodes having a possible path for backtracking the tree if needed. Backtracking is not a linear approach, so we need different data structures for traversing through the whole tree. The stack and queue are the major data structure that is used for traversing a tree.

Traversal is a technique for visiting all of a tree's nodes and printing their values. Traversing a tree involves iterating over all nodes in some manner. We always start from the root (head) node since all nodes are connected by edges (links). As the tree is not a linear data structure, there can be more than one possible next node from a given node, so some nodes must be deferred, i.e., stored in some way for later visiting.

Types of Traversal of Binary Tree

There are three types of traversal of a binary tree.

1. Inorder tree traversal
2. Preorder tree traversal
3. Postorder tree traversal

Inorder Tree Traversal: The left subtree is visited first, followed by the root, and finally the right subtree in this traversal strategy. Always keep in mind that any node might be a subtree in and of itself. The output of a binary tree traversal in order produces sorted key values in ascending order.

Example:

```
//C Program for Inorder traversal of the binary search tree

#include<stdio.h>
#include<stdlib.h>

struct node
{
   int key;
   struct node *left;
   struct node *right;
};

//return a new node with the given value
struct node *getNode(int val)
{
   struct node *newNode;

   newNode = malloc(sizeof(struct node));

   newNode->key   = val;
   newNode->left  = NULL;
   newNode->right = NULL;

   return newNode;
}

//inserts nodes in the binary search tree
struct node *insertNode(struct node *root, int val)
{
    if(root == NULL)
       return getNode(val);

    if(root->key < val)
       root->right = insertNode(root->right,val);

    if(root->key > val)
       root->left = insertNode(root->left,val);

    return root;
}
```

```
//inorder traversal of the binary search tree
void inorder(struct node *root)
{
   if(root == NULL)
      return;

   //traverse the left subtree
   inorder(root->left);

   //visit the root
   printf("%d ",root->key);

   //traverse the right subtree
   inorder(root->right);
}

int main()
{
  struct node *root = NULL;


   int data;
   char ch;
      /*  Do while loop to display various options to select from to decide the input  */
      do
      {
         printf("\nSelect one of the operations::");
         printf("\n1. To insert a new node in the Binary Tree");
         printf("\n2. To display the nodes of the Binary Tree(via Inorder Traversal).\n");

         int choice;
         scanf("%d",&choice);
         switch (choice)
         {
         case 1 :
            printf("\nEnter the value to be inserted\n");
            scanf("%d",&data);
            root = insertNode(root,data);
            break;
         case 2 :
            printf("\nInorder Traversal of the Binary Tree::\n");
            inorder(root);
            break;
         default :
```

```
            printf("Wrong Entry\n");
            break;
        }

        printf("\nDo you want to continue (Type y or n)\n");
        scanf(" %c",&ch);
    } while (ch == 'Y'|| ch == 'y');

  return 0;
}
```

Output:

```
Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree (via Inorder Traversal).
1

Enter the value to be inserted
12

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Inorder Traversal).
1

Enter the value to be inserted
98

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Inorder Traversal).
1

Enter the value to be inserted
23

Do you want to continue (Type y or n)
y
```

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Inorder Traversal).
1

Enter the value to be inserted
78

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Inorder Traversal).
1

Enter the value to be inserted
45

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Inorder Traversal).
1

Enter the value to be inserted
87

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Inorder Traversal).
2

Inorder Traversal of the Binary Tree::
12 23 45 78 87 98
Do you want to continue (Type y or n)
n

Preorder Tree Traversal: In this traversal method, the root node is visited first, then the left subtree, and finally the right subtree.

Example:

```
/*
 * Program: Preorder traversal of the binary search tree
 * Language: C
 */

#include<stdio.h>
#include<stdlib.h>

struct node
{
   int key;
   struct node *left;
   struct node *right;
};

//return a new node with the given value
struct node *getNode(int val)
{
   struct node *newNode;

   newNode = malloc(sizeof(struct node));

   newNode->key   = val;
   newNode->left  = NULL;
   newNode->right = NULL;

   return newNode;
}

//inserts nodes in the binary search tree
struct node *insertNode(struct node *root, int val)
{
    if(root == NULL)
       return getNode(val);

    if(root->key < val)
       root->right = insertNode(root->right,val);

    if(root->key > val)
       root->left = insertNode(root->left,val);

    return root;
```

```
}

//preorder traversal of the binary search tree
void preorder(struct node *root)
{
   if(root == NULL)
      return;

   //visit the root
   printf("%d ",root->key);

   //traverse the left subtree
   preorder(root->left);

   //traverse the right subtree
   preorder(root->right);
}

int main()
{
  struct node *root = NULL;

  int data;
   char ch;
      /*  Do while loop to display various options to select from to decide the input  */
      do
      {
         printf("\nSelect one of the operations::");
         printf("\n1. To insert a new node in the Binary Tree");
         printf("\n2. To display the nodes of the Binary Tree(via Preorder Traversal).\n");

         int choice;
         scanf("%d",&choice);
         switch (choice)
         {
         case 1 :
            printf("\nEnter the value to be inserted\n");
            scanf("%d",&data);
            root = insertNode(root,data);
            break;
         case 2 :
            printf("\nPreorder Traversal of the Binary Tree::\n");
            preorder(root);
            break;
         default :
```

```
            printf("Wrong Entry\n");
            break;
          }

          printf("\nDo you want to continue (Type y or n)\n");
          scanf(" %c",&ch);
       } while (ch == 'Y'|| ch == 'y');

  return 0;
}
```

Output:

```
Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Preorder Traversal).
1

Enter the value to be inserted
45

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Preorder Traversal).
1

Enter the value to be inserted
53

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Preorder Traversal).
1

Enter the value to be inserted
1

Do you want to continue (Type y or n)
y

Select one of the operations::
```

1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Preorder Traversal).

1

Enter the value to be inserted

2

Do you want to continue (Type y or n)

y

Select one of the operations::

1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Preorder Traversal).

1

Enter the value to be inserted

97

Do you want to continue (Type y or n)

y

Select one of the operations::

1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Preorder Traversal).

1

Enter the value to be inserted

22

Do you want to continue (Type y or n)

y

Select one of the operations::

1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Preorder Traversal).

2

Preorder Traversal of the Binary Tree::

45 1 2 22 53 97

Do you want to continue (Type y or n)

y

Select one of the operations::

1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Preorder Traversal).

1

Enter the value to be inserted
76

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Preorder Traversal).
1

Enter the value to be inserted
30

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Preorder Traversal).
1

Enter the value to be inserted
67

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Preorder Traversal).
1

Enter the value to be inserted
4

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Preorder Traversal).
2

Preorder Traversal of the Binary Tree::
45 1 2 22 4 30 53 97 76 67
Do you want to continue (Type y or n)
n

Postorder Tree Traversal: The root node is visited last in this traversal method, hence the name. First, we traverse the left subtree, then the right subtree, and finally the root node.

Example:

```
/*
 * Program: Postorder traversal of the binary search tree
 * Language: C
 */

#include<stdio.h>
#include<stdlib.h>

struct node
{
   int key;
   struct node *left;
   struct node *right;
};

//return a new node with the given value
struct node *getNode(int val)
{
   struct node *newNode;

   newNode = malloc(sizeof(struct node));

   newNode->key   = val;
   newNode->left  = NULL;
   newNode->right = NULL;

   return newNode;
}
//inserts nodes in the binary search tree
struct node *insertNode(struct node *root, int val)
{
    if(root == NULL)
       return getNode(val);

    if(root->key < val)
```

```
        root->right = insertNode(root->right,val);

    if(root->key > val)
        root->left = insertNode(root->left,val);

    return root;
}

//postorder traversal of the binary search tree
void postorder(struct node *root)
{
   if(root == NULL)
      return;

   //traverse the left subtree
   postorder(root->left);

   //traverse the right subtree
   postorder(root->right);

   //visit the root
   printf("%d ",root->key);
}
int main()
{
  struct node *root = NULL;


  int data;
   char ch;
      /*  Do while loop to display various options to select from to decide the input  */
      do
      {
         printf("\nSelect one of the operations::");
         printf("\n1. To insert a new node in the Binary Tree");
         printf("\n2. To display the nodes of the Binary Tree(via Postorder Traversal).\n");

         int choice;
         scanf("%d",&choice);
         switch (choice)
         {
         case 1 :
            printf("\nEnter the value to be inserted\n");
            scanf("%d",&data);
            root = insertNode(root,data);
```

```
            break;
        case 2 :
            printf("\nPostorder Traversal of the Binary Tree::\n");
            postorder(root);
            break;
        default :
            printf("Wrong Entry\n");
            break;
        }

        printf("\nDo you want to continue (Type y or n)\n");
        scanf(" %c",&ch);
    } while (ch == 'Y'|| ch == 'y');

  return 0;
}
```

Output:

```
Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Postorder Traversal).
1

Enter the value to be inserted
12

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Postorder Traversal).
1

Enter the value to be inserted
31

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Postorder Traversal).
24
Wrong Entry
```

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Postorder Traversal).
1

Enter the value to be inserted
24

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Postorder Traversal).
1

Enter the value to be inserted
88

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Postorder Traversal).
1

Enter the value to be inserted
67

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Postorder Traversal).
1

Enter the value to be inserted
56

Do you want to continue (Type y or n)

y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Postorder Traversal).
1

Enter the value to be inserted
90

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Postorder Traversal).
1

Enter the value to be inserted
44

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Postorder Traversal).
1

Enter the value to be inserted
71

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Postorder Traversal).
1

Enter the value to be inserted
38

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Postorder Traversal).
1

Enter the value to be inserted
29

Do you want to continue (Type y or n)
y

Select one of the operations::
1. To insert a new node in the Binary Tree
2. To display the nodes of the Binary Tree(via Postorder Traversal).
2

Postorder Traversal of the Binary Tree::
29 24 38 44 56 71 67 90 88 31 12
Do you want to continue (Type y or n)
n

Question 3: What are AVL trees? How do they differ from Splay trees.

Ans. AVL Tree: AVL Tree is invented by GM Adelson - Velsky and EM Landis in 1962. The tree is named AVL in honour of its inventors. AVL Tree can be defined as height balanced binary search tree in which each node is associated with a balance factor which is calculated by subtracting the height of its right sub-tree from that of its left sub-tree.

Tree is said to be balanced if balance factor of each node is in between -1 to 1, otherwise, the tree will be unbalanced and need to be balanced.

Balance Factor (k) = height (left(k)) - height (right(k))

If balance factor of any node is 1, it means that the left sub-tree is one level higher than the right sub-tree.

If balance factor of any node is 0, it means that the left sub-tree and right sub-tree contain equal height.

If balance factor of any node is -1, it means that the left sub-tree is one level lower than the right sub-tree.

AVL Rotations

We perform rotation in AVL tree only in case if Balance Factor is other than -1, 0, and 1. There are basically four types of rotations which are as follows:

1. L L rotation: Inserted node is in the left subtree of left subtree of A
2. R R rotation : Inserted node is in the right subtree of right subtree of A
3. L R rotation : Inserted node is in the right subtree of left subtree of A
4. R L rotation : Inserted node is in the left subtree of right subtree of A

Where node A is the node whose balance Factor is other than -1, 0, 1.

The first two rotations LL and RR are single rotations and the next two rotations LR and RL are double rotations. For a tree to be unbalanced, minimum height must be at least 2, Let us understand each rotation

1. RR Rotation: When BST becomes unbalanced, due to a node is inserted into the right subtree of the right subtree of A, then we perform RR rotation, RR rotation is an anticlockwise rotation, which is applied on the edge a node having balance factor -2

2. LL Rotation: When BST becomes unbalanced, due to a node is inserted into the left subtree of the left subtree of C, then we perform LL rotation, LL rotation is clockwise rotation, which is applied on the edge a node having balance factor 2.

3. LR Rotation: Double rotations are bit tougher than single rotation which has already explained above. LR rotation = RR rotation + LL rotation, i.e., first RR rotation is performed on subtree and then LL rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.

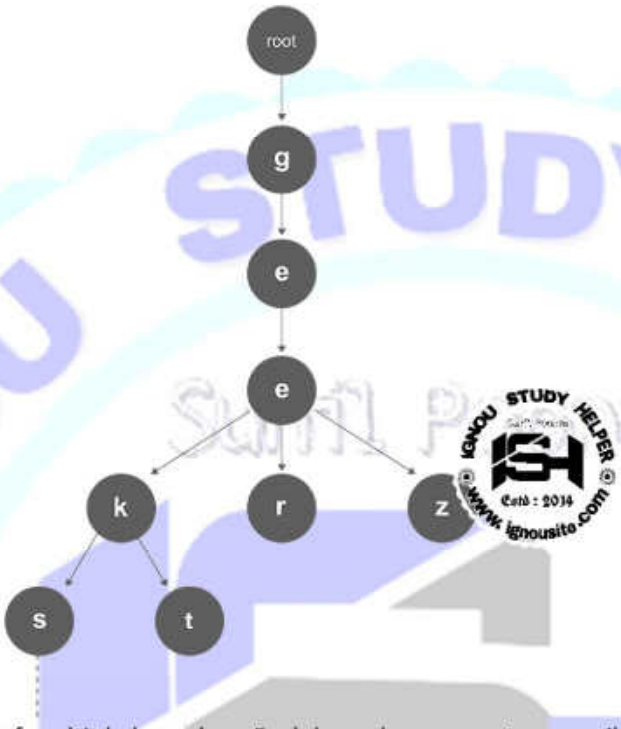
4. RL Rotation: As already discussed, that double rotations are bit tougher than single rotation which has already explained above. R L rotation = LL rotation + RR rotation, i.e., first LL rotation is performed on subtree and then RR rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.

AVL Tree vs Splay Tree:

- AVL Tree has a guaranteed fast runtime (O(log n)) on every lookup, insert and deletion
- Splay tree is faster for repeated lookup
- For real-time applications, AVL Tree is better than Splay tree
- For non real time applications, Splay tree has better average running time
- Splay tree is more efficient in tree merging and splitting
- Splay tree is more memory efficient than AVL tree because it does not need to store balance information in the nodes
- AVL Tree supports multi-threading and parallel lookup while Splay tree cannot because it reshapes it on every lookup
- Splay tree is a better choice if only small number of nodes will be accessed or some nodes are accessed more than others
- Splay tree is easier to implement because the rotation logic is simpler
- For splay tree, any long sequence operations will take at most O(n log n) time and individual operation could take O(n) time.

Question 4: What are Tries? How do they differ from Binary Tries?

Ans. Tries: Tries is an efficient information reTriesval data structure. Using Tries, search complexities can be brought to optimal limit (key length). If we store keys in a binary search tree, a well balanced BST will need time proportional to M * log N, where M is the maximum string length and N is the number of keys in the tree. Using Tries, we can search the key in O(M) time. However, the penalty is on Tries storage requirements.



Every node of Tries consists of multiple branches. Each branch represents a possible character of keys. We need to mark the last node of every key as the end of the word node. A Tries node field isEndOfWord is used to distinguish the node as the end of the word node. A simple structure to represent nodes of the English alphabet can be as follows,

```
// Tries node
struct TriesNode
{
    struct TriesNode *children[ALPHABET_SIZE];
    // isEndOfWord is true if the node
    // represents end of a word
    bool isEndOfWord;
};
```

Inserting a key into Tries is a simple approach. Every character of the input key is inserted as an individual Tries node. Note that the children is an array of pointers (or references) to next level Tries nodes. The key character acts as an index to the array children. If the input key is new or an extension of the existing key, we need to construct non-existing nodes of the key, and mark the end of the word for the last node. If the input key is a prefix of the existing key in Tries, we simply mark the last node of the key as the end of a word. The key length determines Tries depth.

Searching for a key is similar to an insert operation, however, we only compare the characters and move down. The search can terminate due to the end of a string or lack of key in the Tries. In the former case, if the isEndofWord field of the last node is true, then the key exists in the Tries. In the second case, the search terminates without examining all the characters of the key, since the key is not present in the Tries.

The following picture explains the construction of Tries using keys given in the example below,

In the picture, every character is of type Tries_node_t. For example, the root is of type Tries_node_t, and it's children a, b and t are filled, all other nodes of root will be NULL. Similarly, "a" at the next level is having only one child ("n"), all other children are NULL. The leaf nodes are in blue.

Tries vs Binary Tries:

Binary Tries: A binary tries is a dynamic version of what happens during quicksort. The root represents an arbitrary (but hopefully not too far off from the median) pivot element from the collection. The left subtree is then everything less than the root, and the right subtree is everything greater than the root. The left and right collections are then again ordered in the same manner, i.e. the data structure is defined recursively.

Tries: A tries is a dynamic version of what happens during radix sort. You look at the first bit or digit of a number (or first letter of a string) to determine which subtree the value belongs in. You then repeat the procedure recursively using the next character or digit to determine which of the subtree's children it belongs in, and so on.

One way to think about it is that binary search trees take their pivot elements from the collection itself. A trie's pivots, on the other hand, are based on splitting the range of possible inputs evenly. Consider a binary 0-1 tries that stores 32-bit unsigned integers (numbers from 0 to 2^32-1, inclusive). The left subtree contains all the elements of the collection whose most significant bit is 0; the right subtree contains all those whose MSB is 1. This is as though the pivot element was right between 2^31 - 1 and 2^31. It's exactly the pivot needed to split the range of possible integers (0 to 2^32 - 1) into half, with the left side containing 0 to 2^31 - 1 and the right side 2^31 to 2^32 - 1, so exactly 2^31 elements in each range.

Tries are not necessarily binary tries structures, because they can split the input range into any number of parts. A tries node could have, for example, one subtree for each letter in the alphabet.